Lecture 1 — Introduction to Competitive Programming: Judges, Verdicts, I/O

> Year Curriculum · Prerequisites: basic programming in any language (variables, loops, functions, arrays) · Estimated time: 2 hours lecture + 3 hours practice

1. Overview

Competitive programming (CP) is the sport of solving precisely specified algorithmic problems quickly and correctly. You read a problem, design an algorithm, implement it, and submit the source code to an **online judge**. The judge compiles your program, runs it on a set of hidden tests, and compares your output to the expected output. You get credit only if every test passes within the **time limit** and **memory limit**.

This first lecture is about the rules of the game rather than about algorithms. By the end you should be able to:

- explain how the major contest formats (ICPC, IOI-style olympiads, Codeforces, AtCoder) are scored;
- use the online judges we will rely on all year;
- read a problem statement and extract the information that matters, especially the constraints;
- interpret each judge verdict and know its typical causes;
- write correct, fast C++ input/output for the three common input layouts;
- avoid the mistakes that cost beginners most of their wrong answers: overflow, wrong format, and stray output.

The rest of the year builds on these habits. Week 2 covers C++ and the STL; Week 3 makes the "how fast is fast enough" question precise with complexity analysis.

2. Contest formats

ICPC (International Collegiate Programming Contest) is the main university team contest. Teams of three share one computer for five hours and receive roughly 10–13 problems. Teams are ranked by number of problems solved; ties are broken by **penalty time**: for each solved problem, the minutes from contest start to the accepted submission, plus 20 minutes for each rejected submission on that problem. Wrong submissions on problems you never solve cost nothing. The ladder is regional contests → national/continental championships → World Finals, which this curriculum's later tracks cover.

IOI-style contests (national olympiads, IOI, USACO) are individual. Problems are split into **subtasks** with smaller constraints, and you get partial credit for each subtask solved. A slow but correct brute force often earns meaningful points here.

Codeforces rounds last about two hours. Problem values decrease over time, and in some rounds you can "hack" other contestants by giving inputs that break their solutions. Your rating moves after each rated round, which makes it a good way to measure progress.

AtCoder runs Beginner (ABC), Regular (ARC), and Grand (AGC) contests. ABC problems A–D are excellent material for the first half of this course: statements are short and precise, and the tests are strong.

3. The judges we will use

Judge	URL	Why
Codeforces	https://codeforces.com	Huge archive with tags and difficulty ratings; weekly contests
AtCoder	https://atcoder.jp	Clean beginner problems; editorials for every contest
CSES Problem Set	https://cses.fi/problemset	~300 classic problems ordered by topic; our main practice set
Kattis	https://open.kattis.com	ICPC-style archive, including many regional problems
USACO	https://usaco.org	Olympiad training ladder (Bronze, Silver, Gold, Platinum)

Problem IDs in these notes are given in each judge's own scheme: "CF 4A" is Codeforces contest 4, problem A (`codeforces.com/problemset/problem/4/A`); "CSES 1068" is `cses.fi/problemset/task/1068`; "ABC086 A" is AtCoder Beginner Contest 086, task A.

4. Reading a problem statement

A typical statement has: title, time limit (often 1–2 seconds), memory limit (often 256 MB), a story describing the task, the input format, the output format, **constraints**, sample tests, and sometimes notes explaining the samples.

Read in this order:

1. **What is being asked?** Strip the story away and restate the task in one sentence of mathematics, for example: "given an array, output the minimum number of increments that make it non-decreasing."
2. **Constraints.** These decide the algorithm. With `n ≤ 20` you can try all subsets. With `n ≤ 2·10^5` you need roughly `O(n log n)`. Value bounds such as `|a_i| ≤ 10^9` decide the data type: a sum of 2·10^5 such values does not fit in a 32-bit `int`.

3. **Output format.** Is it "YES"/"NO" or "Yes"/"No"? One answer per line, or space separated? Should impossible cases print -1? Is a floating-point answer checked with a tolerance?
4. **Samples.** Work through at least one by hand. If your understanding gives a different answer from the sample, you have misread the problem, and no amount of coding will fix that.

The speed rule of thumb

A modern judge machine performs roughly **10^8 simple operations per second** in optimized C++. The table below follows from it and is the most useful single fact in CP. Week 3 derives it properly.

Largest `n`	Acceptable complexity (≈1 s)
10–11	`O(n!)`
20–25	`O(2^n)`, `O(2^n · n)`
100	`O(n^4)`
500	`O(n^3)`
5,000	`O(n^2)`
2·10^5 – 10^6	`O(n log n)`
10^7 – 10^8	`O(n)`
10^18	`O(log n)` or `O(1)`

5. Verdicts and what they usually mean

- **AC — Accepted.** Every test passed.
- **WA — Wrong Answer.** The output differs from the expected output on some test. Causes, from most to least common: integer overflow; a missed edge case (n = 1, all values equal, an empty answer); misreading the problem; wrong output format; an incorrect algorithm, such as a greedy that is not actually optimal.
- **TLE — Time Limit Exceeded.** The program did not finish in time. The algorithm may be too slow for the constraints, the I/O may be slow (no fast I/O, `endl` in a loop), or the program may be stuck in an infinite loop, for example a `while` whose variable overflowed.
- **MLE — Memory Limit Exceeded.** Too much memory was used. One `int` takes 4 bytes, so `int a[10000][10000]` is 400 MB and is too large. Deep recursion and accidentally copying big containers also cause it.
- **RE — Runtime Error.** The program crashed or returned a non-zero exit code. Causes include out-of-bounds array access, division or modulo by zero, stack overflow from very deep recursion, dereferencing an invalid iterator (such as `*s.begin()` on an empty set), and an uncaught exception.

- **CE — Compilation Error.** Usually the wrong language or compiler was selected, or the code uses a compiler-specific feature. The judge shows the compiler message, so read it.
- **PE — Presentation Error.** Some judges, mostly older ICPC archives, report this when the answer is correct but whitespace differs. Most modern judges ignore trailing spaces and newlines, but not missing separators or extra words.
- **Idleness limit exceeded / wrong output format on interactive problems.** The program waited for input it never received, usually because it did not flush its output. This matters for interactive problems later in the year.

Tests run in order and the first failure is reported. "WA on test 1" almost always means the program fails the first sample, which you could have caught locally.

6. Input/output in C++

6.1 The three input layouts

Single test. Read the input as described:

```
int n;
cin >> n;
vector<long long> a(n);
for (auto &x : a) cin >> x;
```

`cin >> x` skips any whitespace, including spaces, tabs, and newlines. Whether the numbers sit on one line or many does not matter.

Multiple tests with a count. Codeforces in particular uses this layout:

```
int t;
cin >> t;
while (t--) {
    // read and solve one test
}
```

Two rules follow. First, **reset state**: any global array, counter, or container you reuse must be cleared for each test. Second, the statement usually bounds the **sum** of `n` over all tests (for example, "the sum of n does not exceed 2·10^5"). Then an `O(n)` solution per test is fine, but clearing a global array of size 2·10^5 on every test is **not**, because with t = 10^4 tests that is 2·10^9 operations. Clear only the first `n` entries.

Until end of file. Kattis and older ICPC archives often give no test count:

```
long long a, b;
while (cin >> a >> b) {
```

```
    cout << llabs(a - b) << '\n';
}
```

`cin >> a` evaluates to false when the input is exhausted, so the loop stops cleanly. For line-based input, use `getline(cin, s)`. If you read a number with `>>` and then call `getline`, the first `getline` returns the rest of the number's line, which is usually empty. Skip it with `cin.ignore(numeric_limits<streamsize>::max(), '\n')` or with an extra `getline`.

6.2 Fast I/O

By default `cin`/`cout` are synchronized with C's `stdio`, which makes them slow. Put these two lines at the top of `main`:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

The first line turns off synchronization. The second stops `cin` from flushing `cout` before every read. With both, `cin` reads 10^6 integers in well under 0.1 s. After calling them, **do not mix** `cin`/`cout` with `scanf`/`printf`, because their buffers are no longer coordinated.

Use `'\n'` rather than `endl`. `endl` writes a newline and also **flushes** the buffer. Printing 10^6 lines with `endl` can take seconds.

6.3 Output formatting

- Print floating-point values with fixed precision: `cout << fixed << setprecision(9) << x;`. Most problems accept an absolute or relative error of 1e-6, so printing 9 or more digits is safe.
- Print exactly the words the statement asks for. "Yes" and "YES" are different strings to a strict checker.
- Never print prompts ("Enter n:") or debug output to `cout`. Write debug output to `cerr`, which judges ignore, or remove it.

7. Integer overflow

A 32-bit `int` holds values up to 2,147,483,647 ≈ 2.1·10^9, and a `long long` holds up to about 9.2·10^18. Overflow of a signed integer is **undefined behavior** in C++. In practice it wraps to a negative number, but the optimizer may also assume it never happens and produce strange code.

Typical traps:

- **Sums.** Adding 2·10^5 values of size 10^9 gives up to 2·10^14, which needs `long long`.

- **Products.** In `int a = 100000, b = 100000; long long c = a * b;` the multiplication is done in `int` *before* the conversion, so it overflows. Write `1LL * a * b` or declare `a` and `b` as `long long`.
- **Intermediate values.** In the Weird Algorithm (CSES 1068), the start value is at most 10^6 but intermediate values pass 2^31 for some starting points. Squares in distance formulas, `mid * mid` in binary search, and factorials behave the same way.

Default habit: **use `long long` for anything that is a sum, a product, or an answer**, unless memory is tight. The cost is negligible.

8. Worked examples

8.1 CSES 1068 — Weird Algorithm

The problem simulates the Collatz process from `n` and prints every value. It is a direct simulation. The only difficulty is overflow, which `long long` fixes. Complexity: the number of steps is small (a few hundred for n ≤ 10^6), so time is not an issue.

8.2 CSES 1083 — Missing Number

The input contains all numbers 1..n except one. Read `n` and then `n - 1` numbers. The sum 1 + 2 + … + n = n(n+1)/2 can reach about 2·10^10, so use `long long`. Subtract each number read; what remains is the missing one. Time `O(n)`, memory `O(1)`. An alternative that cannot overflow at all: XOR all of 1..n with all the given numbers. Equal values cancel, leaving the missing number (bit tricks come in Week 9).

8.3 CSES 1069 — Repetitions

Given a DNA string, find the length of the longest run of one repeated character. Scan once, keeping the current run length and the best so far:

```
for i in 1..|s|-1:
    if s[i] == s[i-1]: cur++ else cur = 1
    best = max(best, cur)
```

Initialize `cur = best = 1`, because the string is non-empty. Starting `best` at 0 breaks on strings of length 1, the classic off-by-one.

8.4 CSES 1094 — Increasing Array

Find the minimum total amount you must add to elements to make the array non-decreasing. Greedy: walk left to right. Whenever `a[i] < a[i-1]`, raise `a[i]` to `a[i-1]` and add the difference to the answer. Raising it any higher only costs more, both now and for later elements, so this is optimal. The answer can reach about 2·10^5 · 10^9, so use `long long`.

8.5 Codeforces 4A — Watermelon

Decide whether `w` can be split into two positive even parts. Most people first answer "YES iff `w` is even" and fail on `w = 2`. The correct condition is `w % 2 == 0 && w > 2`. Lesson: before submitting, try the **smallest** inputs by hand.

9. Reference implementations

9.1 Weird Algorithm (CSES 1068)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long n;
    cin >> n;
    cout << n;
    while (n != 1) {
        if (n % 2 == 0) n /= 2;
        else n = 3 * n + 1;
        cout << ' ' << n;
    }
    cout << '\n';
    return 0;
}
```

9.2 Increasing Array (CSES 1094)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
```

```
    for (auto &x : a) cin >> x;
    long long moves = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] < a[i - 1]) {
            moves += a[i - 1] - a[i];
            a[i] = a[i - 1];
        }
    }
    cout << moves << '\n';
    return 0;
}
```

9.3 Multiple tests and read-until-EOF skeletons

```
#include <bits/stdc++.h>
using namespace std;

// Pattern A: t test cases. Each test: n, then n numbers; print their sum.
void solveWithCount() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        long long sum = 0;
        for (int i = 0; i < n; i++) {
            long long x;
            cin >> x;
            sum += x;
        }
        cout << sum << '\n';
    }
}

// Pattern B: pairs until EOF; print |a - b| for each.
void solveUntilEOF() {
    long long a, b;
    while (cin >> a >> b) cout << llabs(a - b) << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    solveWithCount();   // swap for solveUntilEOF() as needed
    return 0;
}
```

10. A workflow for every problem

1. Read the full statement. Restate it in one sentence. Note the constraints.

2. Solve the samples by hand.
3. Find an algorithm and check its complexity against the speed table.
4. Write the code, starting from your template (next week).
5. Compile with warnings and sanitizers while testing: `g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=address,undefined -o a a.cpp`
6. Run the samples: `./a < in1.txt`. Compare automatically with `diff <(./a < in1.txt) out1.txt`.
7. Try edge cases: minimum `n`, maximum values, all elements equal, already-sorted input.
8. Submit. If you get anything other than AC, **re-read the statement first**, then check overflow, then check edge cases, and only then look for deeper logic bugs.

11. Common mistakes

- **Overflow.** Using `int` for sums or products, or computing `a * b` in `int` and then storing it in `long long`.
- **Extra output.** Prompts, debug prints, or stray spaces in a strict format.
- **`endl` in large loops.** Costs TLE for no reason. Use `'\n'`.
- **Not resetting between tests.** Global counters, `visited` arrays, and containers keep stale values from the previous test.
- **Clearing too much between tests.** `memset` on a 10^6-sized array for each of 10^4 tests is 10^10 operations.
- **Uninitialized variables.** Local variables in C++ are not zero-initialized, so `int best;` holds garbage. Global arrays are zero-initialized.
- **Wrong case in "YES"/"Yes".**
- **Off-by-one on sizes.** Reading `n` numbers when the input has `n - 1`, as in Missing Number.
- **Wrong language selected on the judge.** "GNU C++17" vs. "C11" vs. "Python".
- **Ignoring the sum-of-n guarantee** and allocating `O(maxN)` work per test.

12. Practice problems

Aim for **10 accepted submissions** this week, on at least three different judges.

CSES Introductory Problems

- CSES 1068 — Weird Algorithm — https://cses.fi/problemset/task/1068
- CSES 1083 — Missing Number — https://cses.fi/problemset/task/1083
- CSES 1069 — Repetitions — https://cses.fi/problemset/task/1069
- CSES 1094 — Increasing Array — https://cses.fi/problemset/task/1094

Codeforces

- 4A — Watermelon — https://codeforces.com/problemset/problem/4/A
- 71A — Way Too Long Words — https://codeforces.com/problemset/problem/71/A

- 231A — Team — https://codeforces.com/problemset/problem/231/A
- 158A — Next Round — https://codeforces.com/problemset/problem/158/A
- 282A — Bit++ — https://codeforces.com/problemset/problem/282/A

AtCoder

- ABC086 A — Product — https://atcoder.jp/contests/abc086/tasks/abc086_a
- ABC081 A — Placing Marbles — https://atcoder.jp/contests/abc081/tasks/abc081_a
- ABC081 B — Shift only — https://atcoder.jp/contests/abc081/tasks/abc081_b

Kattis (these practise the read-until-EOF and exact-format habits)

- hello — https://open.kattis.com/problems/hello
- carrots — https://open.kattis.com/problems/carrots
- r2 — https://open.kattis.com/problems/r2

13. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 1 ("Introduction"): free PDF at https://cses.fi/book/book.pdf
- Steven Halim and Felix Halim, *Competitive Programming 4*, Chapter 1 (contest formats, tips, I/O patterns)
- Codeforces blog "Fast I/O in C++" discussions, and the cppreference pages for `std::ios_base::sync_with_stdio` and `std::basic_ios::tie`
- ICPC rules overview: https://icpc.global/
- USACO Guide, "Getting Started" and "Input & Output" modules: https://usaco.guide/